

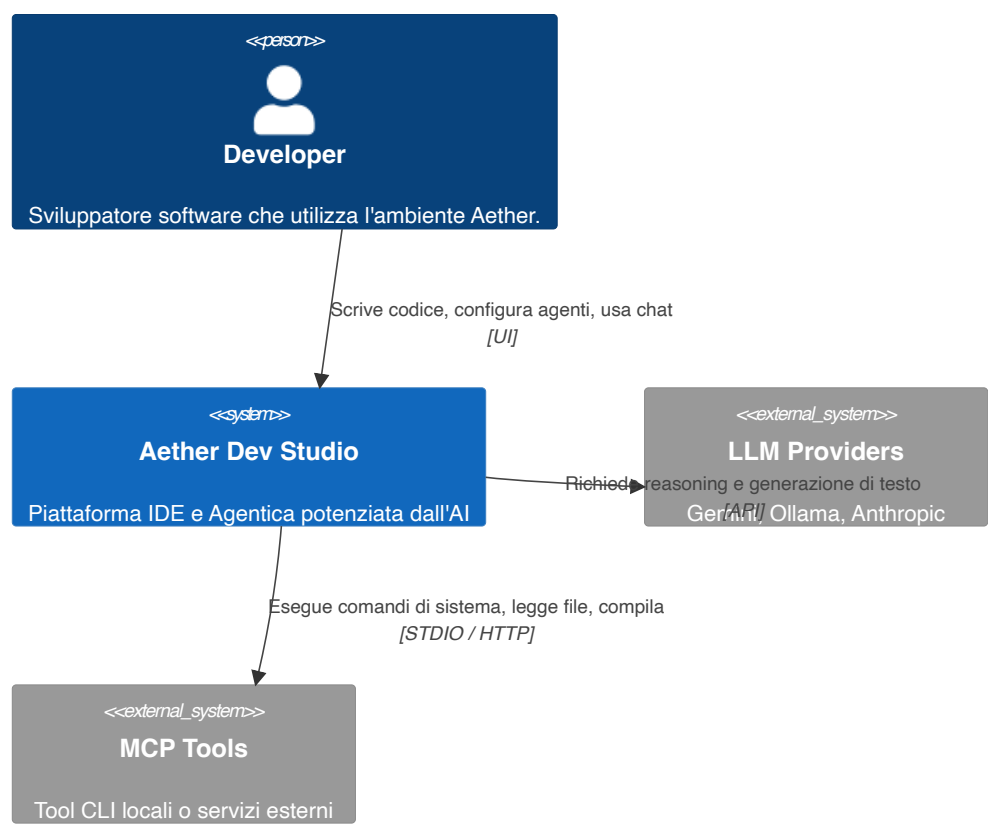
Audit Completo: Aether Dev Studio

Questo documento espone i risultati dell'analisi architetturale end-to-end della codebase di Aether Dev Studio (basato sul branch attuale che integra la persistenza SQLite).

Diagrammi Architeturali (C4 Model)

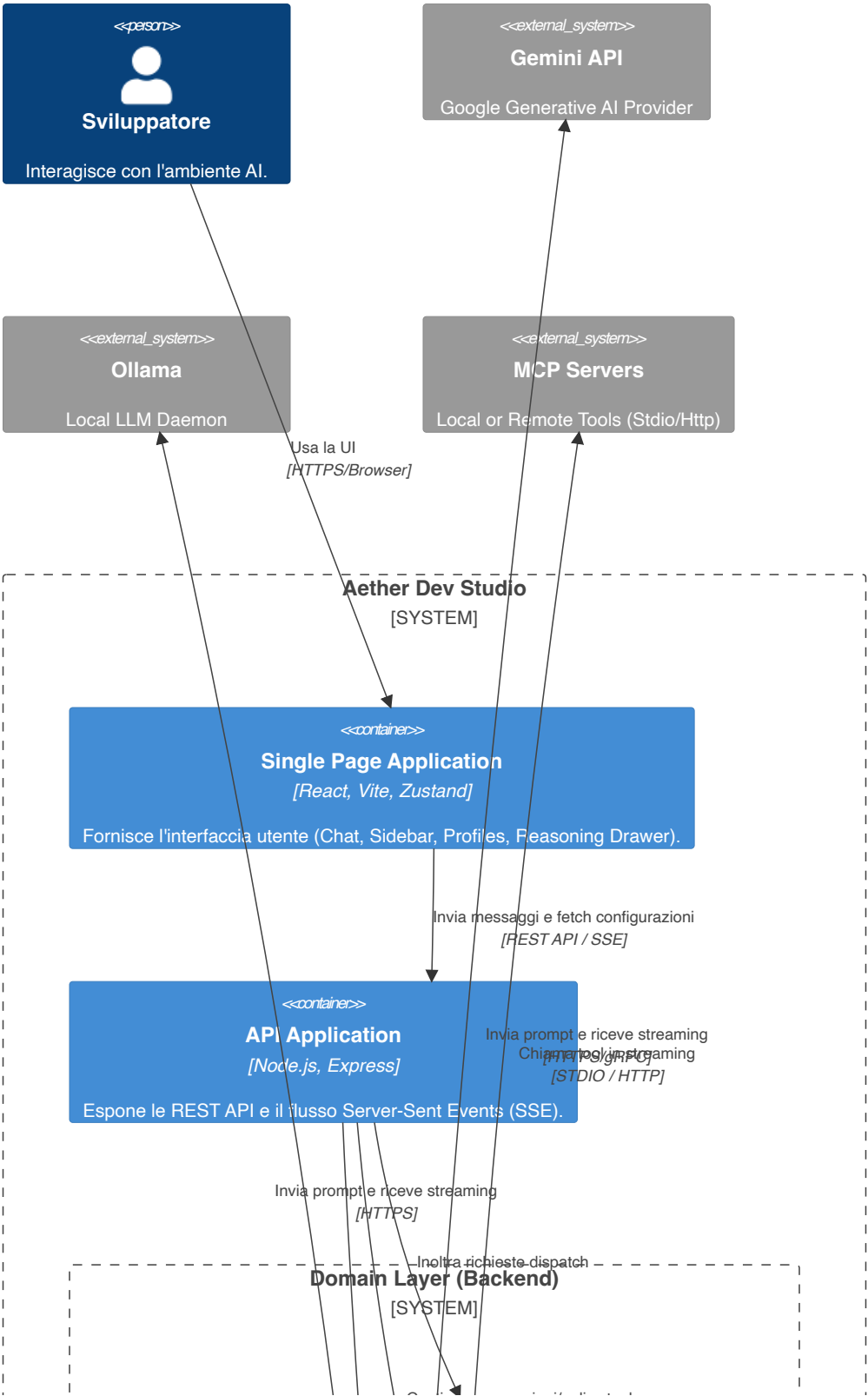
1. System Context Diagram

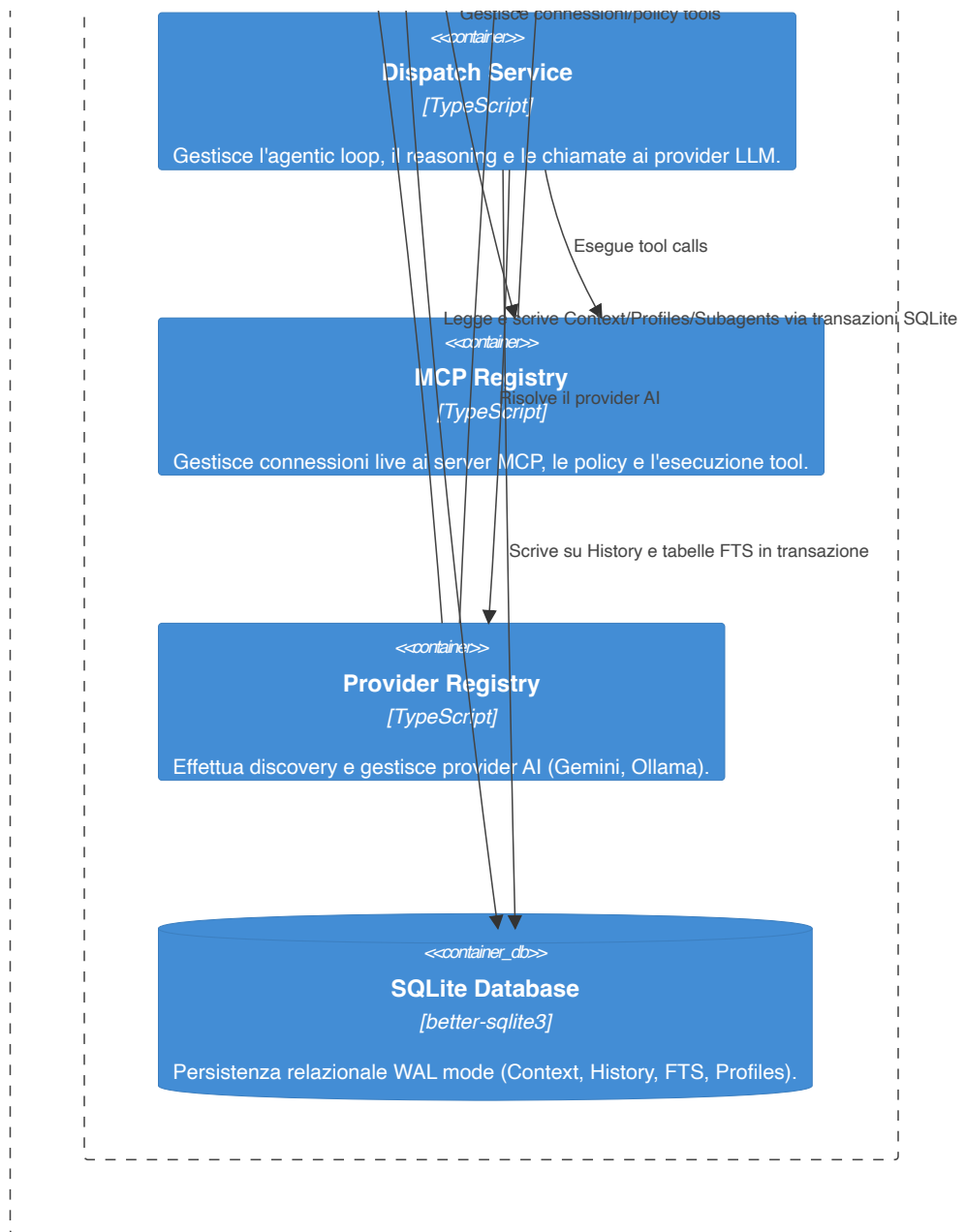
System Context diagram for Aether Dev Studio



2. Container Diagram

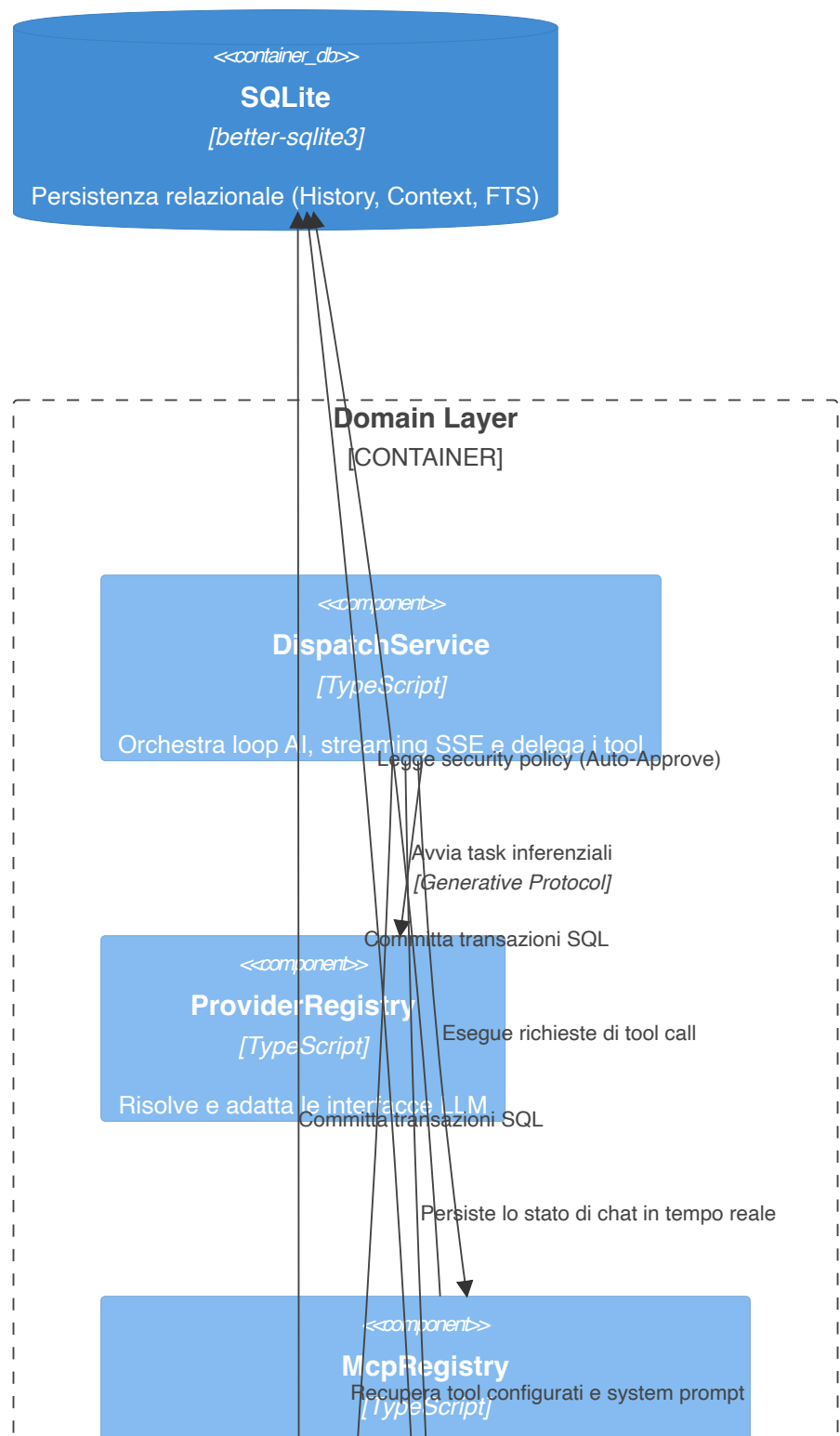
Container diagram for Aether Dev Studio

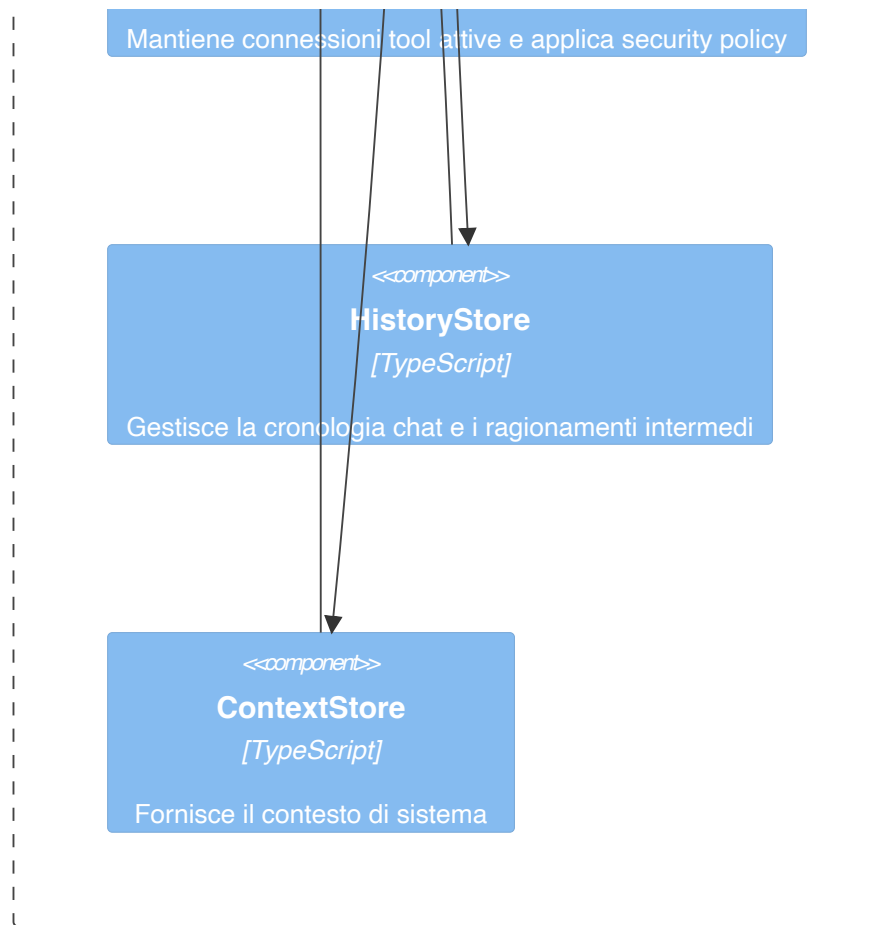




3. Component Diagram (Backend Domain)

Component diagram for Aether Backend Domain





1.1 Store Management (Zustand)

La gestione dello stato client-side (`src/stores`) è stata frammentata seguendo il principio di singola responsabilità (SRP).

- **`useSessionsStore.ts`** : Gestisce metadati e selezione delle sessioni. Utilizza un eccellente pattern di *hydration token* asincrono (`let hydrationToken = 0`) per prevenire race conditions qualora l'utente cambi sessione rapidamente mentre i fetch di storico sono ancora in volo.
- **`useChatStore.ts`** : Gestisce lo stream attivo. Usa mutation ottimistiche dell'array `messages` . L'impatto sul render è ridotto dal momento che il componente React si iscrive selettivamente o tramite memorizzazione.
- **`useMcpStore.ts`** : Traccia le tool calls "in-flight" fornendo feedback real-time su UI senza inquinare la chat principale.

1.2 Event-Driven Hooks

Il file `useStreamingDispatch.ts` agisce come orchestratore per le Server-Sent Events (SSE).

- **Punto di forza:** Il parser SSE non gestisce la rete cruda, ma riceve eventi tipizzati dal layer api (`createStreamingDispatch`). Gli switch sugli eventi (`text`, `thinking`, `reasoning_step`, `tool_call_request`, ecc.) decodificano la semantica complessa (come il function calling progressivo) in update immediati su Zustand.
- **AbortController:** Integrato in maniera corretta, intercettando gli `AbortError` per marcare stream come interrotti anziché falliti.

2. Backend: Domain Driven Design

2.1 Architettura a Livelli

La cartella `server/domain/` divide l'applicazione in bounded contexts (`context`, `dispatch`, `history`, `mcp`, `profiles`, `providers`, `reasoning`, `subagents`).

- Nessun dominio "incrocia" le logiche di database direttamente, ma sfrutta interfacce o Store isolati (es. `ContextStore`, `HistoryStore`).

2.2 Il Motore di Dispatch

Il `DispatchService` implementa il vero "cervello" agentico.

- **Function Calling Iterativo:** Re-instrada ciclicamente l'output dei tool (eseguiti da `McpRegistry`) al provider LLM corrente (es. Gemini o Ollama) mantenendo vivo il controller SSE verso il client. È protetto da loop infiniti tramite costanti empiriche (es. `MAX_TOOL_CALLS_PER_DISPATCH = 10`).

3. Backend: Infrastruttura e API

3.1 SQLite e Sicurezza Dati

- **better-sqlite3** : Ottima scelta per un ambiente monoprocesso Node.js (I/O sincrono in C++ binding evita il disastro di callback-hell o memory leaks sulle code Promises pre-esistenti).

- **FTS (Full-Text Search):** L'integrazione nativa di SQLite per `messages_fts` e transazioni esplicite `db.transaction()` garantiscono velocità e nessuna possibilità di messaggi isolati a causa di shutdown imprevisti.

3.2 Rotte Express e SSE

In `dispatch.routes.ts`, l'evento di chiusura del client è gestito intelligentemente:

```
res.on('close', () => {  
  if (!res.writableEnded) controller.abort();  
});
```

Questo dettaglio previene memory leaks e fermo-macchina inutile sul server qualora l'utente chiuda la tab del browser o cambi sessione. Il server stronca tempestivamente il task in background (incluso l'SDK Anthropic o Gemini).

4. Testing & Qualità (QA)

- **Unità / Integrazione:** L'uso di MSW (`msw-handlers.ts`, `msw-server.ts` in `src/test`) evidenzia che i test Frontend girano completamente in mock sulle risposte SSE (cosa notoriamente difficile da testare, gestita egregiamente).
- **Server DB Tests:** Esiste una suite `server/test` (`db.integration.test.ts`, `test-db.ts`) che monta un server test `:memory:` SQLite per testare `HistoryStore` isolatamente.

5. Configurazione e Tooling

- **Vite & Tailwind:** Configurazione `vite.config.ts` snella.
- **Ottimizzazione Dev Environment:** Disabilitazione intelligente di HMR (`hmr: process.env.DISABLE_HMR !== 'true'`) in situazioni in cui ricaricare l'app interferirebbe con la sessione LLM (utile in ambienti con AI-agents interattivi sulla stessa base di codice).
- **Esbuild:** Compilazione server separata ultra-rapida in `dist/server.cjs`.

Suggerimenti di Refactoring (Actionable Items)

1. **Zod per validazione SSE:** Sebbene ci sia validazione stretta sui payload API e sul database, le risposte SSE dipendono dalla corretta tipizzazione server-to-client. Includere schema Zod condivisi per intercettare disallineamenti di contract.
2. **Rate Limiting Provider:** Dato l'uso esteso di Tool Calling, si suggerisce un backoff/rate-limiter globale per provider (es. 429 su Gemini) a livello di `ProviderRegistry` .